Protocolo individual

Nombre del Estudiante:	Jesús David Díaz Palencia
Código Estudiantil:	7502620080
Asignatura:	Fundamentos de Programación - Unidad 1
Programa / Facultad:	Ingeniería de Software – CTEV, Universidad de Cartagena (2026-2)

Descripción del texto o actividad a realizar

El presente protocolo individual sintetiza la lectura crítica, el análisis conceptual y la apropiación temática de la Unidad 1 del curso Fundamentos de Programación de la Universidad de Cartagena, titulada 'Fundamentos de Programación'. El estudio aborda los pilares esenciales requeridos para la construcción de software: lógica de programación, tipos de datos, operadores lógicos y aritméticos, representación mediante pseudocódigo, algoritmos, estructuras de datos, estructuras de control del flujo de ejecución (selección e iteración) y la modularización mediante funciones y procedimientos.

Palabras claves

Lógica de Programación, Algoritmos, Pseudocódigo, Tipos de Datos, Operadores Aritméticos y Lógicos, Estructuras de Control, Condicionales (Si-Entonces), Bucles (Mientras, Para, Repetir), Modularización, Funciones y Procedimientos.

Objetivos de las lecturas o actividad a realizar

- Comprender los principios fundamentales de la lógica de programación como herramienta analítica para la resolución de problemas mediante algoritmos.
- Identificar y clasificar los tipos de datos primarios (enteros, reales, cadenas, booleanos) y la aplicación correcta de operadores aritméticos y lógicos.
- Diferenciar la representación sintáctica y semántica de las estructuras de control condicionales (If/Else, Según) y repetitivas (For, While, Repetir).
- Evaluar la importancia de la modularización de código a través del uso de funciones y procedimientos para la reutilización y mantenibilidad del software.
- Aplicar los conceptos teóricos construyendo algoritmos en pseudocódigo y verificando su lógica formal mediante entornos de simulación como PSeInt.

Conceptos claves y definiciones

Lógica de Programación: Habilidad analítica para descomponer problemas complejos en subproblemas más simples, diseñando una secuencia ordenada de pasos lógicos para su resolución.

Algoritmo: Conjunto finito, ordenado y no ambiguo de instrucciones o pasos precisos que transforman un conjunto de datos de entrada (input) en un resultado deseado (output).

Pseudocódigo: Lenguaje de especificación de algoritmos intermedio entre el lenguaje humano y un lenguaje de programación formal, diseñado para centrarse en la lógica sin preocuparse por la sintaxis estricta.

Tipos de Datos: Clasificación de los valores que manipula un programa (Enteros, Reales, Cadenas de caracteres, Booleanos), determinando las operaciones válidas sobre ellos y su espacio en memoria.

Operadores Aritméticos y Lógicos: Símbolos especiales que ejecutan operaciones matemáticas (+, -, *, /, %, MOD) o comparaciones lógicas (Y/AND, O/OR, NO/NOT) devolviendo valores verdaderos o falsos.

Estructuras de Control de Selección: Instrucciones condicionales (Si-Entonces-SiNo, Según/Switch) que alteran el flujo lineal de ejecución tomando decisiones basadas en la evaluación de expresiones booleanas.

Estructuras de Control Repetitivas (Bucles): Mecanismos de control de flujo (Para/For, Mientras/While, Repetir/Repeat) que ejecutan iterativamente un bloque de código según condiciones prefijadas o evaluadas en tiempo de ejecución.

Funciones y Procedimientos: Bloques de código independientes y reutilizables que ejecutan tareas específicas. Las funciones retornan un valor explícito al código llamante, mientras que los procedimientos ejecutan una secuencia de acciones sin retornar un valor directo.

Resumen de la lectura

La Unidad 1 de Fundamentos de Programación establece la piedra angular sobre la cual se erige todo el conocimiento técnico en la carrera de Ingeniería de Software. La unidad inicia enfatizando que programar no consiste únicamente en escribir líneas de código en un lenguaje determinado, sino en desarrollar el pensamiento computacional y la lógica deductiva para descomponer problemas reales en algoritmos estructurados.

En primera instancia, el texto define los componentes primarios del procesamiento de información: los tipos de datos (enteros, reales, texto y lógicos) y los operadores que permiten su manipulación. La correcta definición de variables y la aplicación rigurosa de operadores aritméticos (como la división entera o el operador módulo %) y lógicos (que evalúan expresiones complejas combinando condiciones con Y y O) garantizan la integridad de los cómputos computacionales.

Posteriormente, la lectura profundiza en la representación de algoritmos mediante pseudocódigo. El pseudocódigo permite al estudiante plasmar soluciones estructuradas sin las barreras sintácticas de un compilador, sirviendo de puente directo hacia la codificación en lenguajes formales (como Python, C++ o Java).

Un eje central de la unidad son las estructuras de control. Se explica que el flujo lineal por defecto de un programa se modifica mediante dos familias de estructuras: las de selección (condicionales simples, dobles y múltiples) y las de iteración. En el ámbito repetitivo, se contrasta la estructura 'Para' (utilizada cuando el número de iteraciones se conoce a priori) frente a los bucles 'Mientras' y 'Repetir-

Hasta Que' (que dependen de condiciones lógicas que cambian durante la ejecución).

Finalmente, el documento aborda la modularización como principio de diseño de software. El uso de funciones y procedimientos permite dividir programas extensos en componentes independientes, facilitando la legibilidad, la depuración, el trabajo colaborativo y la reutilización del código en múltiples partes de la aplicación.

Metodología de trabajo (Cómo realizó la actividad)

La elaboración del presente protocolo individual se realizó siguiendo una metodología de estudio autorregulado e investigación aplicada:

- Fase 1. Lectura Analítica y Conceptualización: Se realizó la lectura minuciosa de la Unidad 1 de Fundamentos de Programación, identificando los ejes temáticos clave, subrayando las definiciones teóricas y extrayendo las referencias bibliográficas institucionales.
- Fase 2. Experimentación y Validación en PSeInt: Para consolidar los conceptos de estructuras de control (Para, Mientras, Repetir, Según) y manipulación de datos, se construyeron y simularon 4 problemas prácticos en pseudocódigo utilizando la herramienta PSeInt (Validador de Clave, Tabla de Multiplicar, Cajero Automático y Contador Par/Impar).
- Fase 3. Sintesis y Redacción Académica: Se estructuraron las secciones del protocolo conforme a la plantilla institucional de la Universidad de Cartagena, organizando las ideas con rigor técnico y claridad pedagógica.

Conclusiones de la lectura o actividad

- La lógica de programación y el diseño de algoritmos son independientes de cualquier lenguaje específico; dominar las estructuras de control y la abstracción de problemas es la competencia técnica más crucial para un ingeniero de software.
- La selección adecuada de la estructura de repetición (Para vs. Mientras vs. Repetir) responde a la naturaleza del problema: 'Para' es óptimo para iteraciones contadas, mientras que 'Mientras' y 'Repetir' son indispensables para eventos condicionados en tiempo de ejecución.
- La validación defensiva de datos y el manejo de operadores lógicos previenen errores en tiempo de ejecución (runtime errors) y garantizan que los algoritmos respondan de manera predecible ante entradas no válidas.
- La modularización a través de funciones y procedimientos constituye el primer paso hacia arquitecturas de software mantenibles, escalables y orientadas a objetos.

Discusiones y recomendaciones

A partir del estudio realizado, se propone profundizar en las siguientes líneas dentro del plan de estudios de la carrera:

- Fomentar el uso sistemático de diagramas de flujo (ANSI/ISO) junto al pseudocódigo para fortalecer la representación visual de la lógica algorítmica.
- Incorporar tempranamente conceptos de complejidad algorítmica (Notación Big-O) para evaluar

la eficiencia en tiempo y espacio de los algoritmos diseñados.

• Recomendar la práctica continua en entornos de desarrollo interactivos y la transición gradual desde PSeInt hacia lenguajes de alto nivel ampliamente utilizados en la industria como Python y Java.

Bibliografía

• Benavides Acuña, A. (2026). Módulo Unidad 1: Fundamentos de Programación. Universidad de Cartagena.

• Dobrushkin, Vladimir A. (2012). Métodos para análise de algoritmos. Rio de Janeiro: LTC.

• Joyanes Aguilar, L. (2008). Fundamentos de Programación: Algoritmos, Estructuras de Datos y Objetos (4ta ed.). McGraw-Hill.

• Lambert, Kenneth A. (2022). Fundamentos de Python: estruturas de dados. São Paulo: Cengage Learning Brasil.

• PSeInt Software. (2025). Herramienta de asistencia para la enseñanza de la programación y diseño algorítmico. https://pseint.sourceforge.net/